Outlier-Aware Ensemble Selection for Robust Parkinson's Disease Prediction

Month 1 Progress Report

Parv

Supervisor: Prof. K. R. Sahasranand

Department of Data Science, IIT Palakkad

July 2026

Abstract

This report documents Month 1 of a four-month project on outlier-aware ensemble selection for Parkinson's disease (PD) prediction using the Parkinson's Progression Markers Initiative (PPMI) dataset. The objective for this month was *not* the research contribution itself, but a reproducible baseline pipeline that Month 2 will extend. Twelve candidate models (four architectures across three feature modalities) were trained and evaluated, together with five baseline ensemble rules. Two distinct label-leakage problems were identified and corrected during development; both are documented here because they materially affect how the reported numbers should be interpreted. The pipeline is configuration-driven, unit-tested, and fully reproducible from a single random seed.

1 Scope and objectives

The project proposes improving PD prediction by building an ensemble of diverse machine learning models and applying a robust model-selection strategy inspired by *Principled Out-of-Distribution Detection via Multiple Testing* (Magesh, Veeravalli, Roy and Jha, JMLR 2023). Rather than simple majority voting or averaging, validation predictions are treated as structured vectors from which "outlier models" can be identified and removed or down-weighted before aggregation.

Month 1 deliberately excludes that contribution. Its objective is a clean, reproducible baseline pipeline: data finalisation, feature engineering, a multi-modality training pipeline, evaluation, naive ensembles, experiment tracking, configuration, visualisation and documentation. No conformal prediction, Benjamini–Hochberg/Yekutieli procedure, outlier detection or robust aggregation is implemented at this stage; those belong to Month 2.

2 Deliverable status

Deliverable	Status	Location
Data finalisation pipeline	Complete	`src/data/`
Dataset builder (patient-level, seeded, manifest)	Complete	`src/data/splitter.py`
Feature engineering — clinical	Complete	23 raw columns
Feature engineering — imaging	Complete	9 DaTscan SBR columns

Deliverable	Status	Location
Feature engineering — biosignal	Blocked	code written; source not downloaded
Feature engineering — combined	Complete	`src/features/registry.py`
Training pipeline	Complete	`src/training/trainer.py`
Multi-modality training	Complete for available data	12 of a planned 16 runs
Evaluation suite	Complete	`src/evaluation/`
Baseline ensembles	Complete	5 rules
Experiment tracking	Complete	`src/training/`
Configuration system	Complete	`config/config.yaml`
Visualisation	Complete[†]	`src/visualization/plots.py`
Documentation	Complete	README, diagrams, this report

Table 1: Month 1 deliverable status. [†]One documented third-party limitation; see Section 7.

Month 1 is complete for everything the available data permits. The single outstanding functional deliverable — the biosignal modality — is blocked on a dataset that has not yet been downloaded, not on engineering effort. Its code path is written and registered; enabling it is a configuration change.

3 Dataset and experimental design

3.1 Population

The in-distribution population is drawn from the current-era PPMI Clinical curated data cut: sporadic PD versus healthy control (HC), baseline visit only. This yields **1,410 patients** (1,080 sporadic PD, 330 HC) from an initial 19,450 rows across 212 columns.

3.2 Splits

Splitting is performed at the *patient* level and stratified by label, with a seeded, saved manifest recording exactly which patients fall in each split.

Split	Patients	Positive rate	Purpose
Train	846	0.766	model fitting
Validation	211	0.768	model selection, ensemble weights
Calibration	211	0.763	reserved for Month 2 conformal step
Test	142	0.768	final reporting only

Table 2: Patient-level split composition (60/15/15/10, seed 42).

Although the current population contains one row per patient, splitting is implemented at patient level and asserts that no patient appears in two splits. This keeps the guarantee correct if a future longitudinal, multi-visit population is introduced.

3.3 Feature modalities

- **Clinical** (23 columns): demographics plus non-motor, cognitive and behavioural assessments — MoCA, HVLT, Trail Making, Symbol Digit, QUIP, GDS, STAI, SCOPA-AUT and MDS-UPDRS Part I.

- **Imaging** (9 columns): DaTscan striatal binding ratio (SBR) measures for caudate, putamen and striatum.
- **Combined** (32 columns): concatenation of the above.
- **Biosignal**: gait and arm-swing measures. Not yet downloaded; filed under *Motor Assessments* in the PPMI portal rather than the separate *Digital Sensor* category.

4 Data leakage: two findings

Two distinct leakage problems were identified during development. Both are reported here rather than buried, because each materially changes how the results should be read.

4.1 Finding 1: diagnostic circularity in clinical features

An initial training run produced approximately 99–100% AUC-ROC across every model, which is implausible for this task. Investigation showed that Hoehn & Yahr stage and MDS-UPDRS Parts II, III and total were acting as proxies for the diagnosis itself: in PPMI's enrolment protocol, a qualifying motor-exam finding is part of how a participant is classified as PD, and healthy controls are essentially always Hoehn & Yahr stage 0. Median UPDRS-III in this data is 1 for HC against 21 for PD.

Training on those columns therefore does not predict PD — it recovers the enrolment rule. They were removed. MDS-UPDRS Part I was retained: it is patient-reported, not part of the diagnostic examination, and shows a real but much smaller group separation.

A weaker version of the same caveat applies to imaging. PPMI's PD cohort required an abnormal DaTscan for enrolment (a normal scan routes a participant to the separate SWEDD cohort), so strong DaTscan performance is partly expected rather than novel. It was retained because, unlike the motor examination, DaTscan is an independent biological measurement rather than a restatement of the clinical assessment.

4.2 Finding 2: imputation computed across split boundaries

A subsequent code review found that missing-value imputation statistics were computed over the *entire population before splitting*. Measured directly on this dataset, the median used to fill missing BMI values was 26.213 (full population) where the correct training-only value is 26.366.

The absolute difference is small; the methodological consequence is not. Held-out distribution information was entering the training features, so the reported test scores were not clean held-out estimates, and the effect compounds across every imputed column.

The data layer was restructured around an explicit contract:

1. feature builders return *raw* frames with missing values intact;
2. `FeatureCleaner.fit()` observes *training rows only*, learning per-column medians and modes and deciding which columns receive a missingness indicator;
3. `transform()` applies those fixed statistics unchanged to validation, calibration and test;
4. the output column schema is pinned at fit time, so a category absent from a small split cannot silently reshape the feature matrix.

A regression test uses a fixture whose training and held-out medians differ by two orders of magnitude, so any reintroduction of this bug fails loudly rather than quietly.

4.3 Selection discipline

Model selection and ensemble weighting use the validation split only. The test split is evaluated once, for final reporting.

5 Results

5.1 Individual models

Train AUC is reported alongside held-out performance because several models are visibly overfit; the training pipeline now emits a warning when the train–test AUC gap exceeds 0.15.

Run	Train AUC	Test AUC	Test acc.	Test F1
combined + SVM	0.994	**0.984**	0.958	0.973
combined + RandomForest	1.000	0.984	0.944	0.962
imaging + RandomForest	1.000	0.982	0.937	0.958
combined + XGBoost	1.000	0.982	0.944	0.962
imaging + XGBoost	1.000	0.971	0.937	0.958
imaging + NeuralNet	0.978	0.968	0.916	0.944
imaging + SVM	0.990	0.965	0.923	0.949
combined + NeuralNet	0.964	0.954	0.880	0.921
clinical + XGBoost	1.000	0.809	0.824	0.891
clinical + RandomForest	1.000	0.763	0.810	0.887
clinical + SVM	0.919	0.736	0.761	0.856
clinical + NeuralNet	0.698	0.707	0.782	0.875

Table 3: Individual model performance, ranked by test ROC-AUC ($n = 142$).

5.2 Baseline ensembles

Method	Acc.	Prec.	Rec.	F1	ROC-AUC	PR-AUC
Simple average / soft voting*	0.951	0.990	0.945	0.967	**0.9894**	0.9972
Accuracy-weighted average	0.951	0.990	0.945	0.967	0.9889	0.9970
Majority vote	0.944	0.963	0.963	0.963	0.9862	0.9945
Single best model	0.944	0.990	0.936	0.962	0.9817	0.9950

Table 4: Baseline ensembles over all 12 models, test split ($n = 142$). *Simple averaging, soft voting and equal-weighted averaging are mathematically identical for binary classification and are reported as one row.

5.3 Interpretation

Three observations should be carried into the eventual paper.

1. **Averaging beats the single best model**, 0.9894 against 0.9817 ROC-AUC. The ensemble premise holds, but the margin is roughly 0.008 AUC. Month 2's method therefore faces a strong baseline, not a weak one; this should be stated plainly rather than presented as a large opportunity.
2. **Imaging dominates clinical features**, 0.98 against 0.71–0.81 ROC-AUC. Given the enrolment caveat in Section 4, this is partly expected rather than a novel finding.

3. **Several models are heavily overfit**, reaching train AUC 1.000. This became visible only once train-split metrics were added, and should be addressed before Month 2 builds selection logic on these models' validation behaviour.

6 Engineering

6.1 Architecture

The codebase is organised into five layers with an acyclic import graph: foundation (`config`, `utils`), implementations (feature builders, model wrappers), domain (registries, cleaning, splitting, evaluation), orchestration (trainer, ensembler, visualiser) and CLI stages. No module imports a layer above it.

Two extension points keep the architecture open for Month 2 without modification: a new feature modality is added by subclassing `FeatureBuilder` and registering it; a new architecture by subclassing `ModelWrapper` and registering it. Neither requires changes to the training loop, ensembling or visualisation, all of which iterate over whatever is available.

6.2 Reproducibility

- A single seed drives Python's `random`, NumPy, every estimator and the split.
- No paths, feature lists, ratios or hyperparameters are hardcoded; all live in `config.yaml`, validated on load.
- Every run writes a manifest recording hyperparameters, metrics on all four splits, feature set, seed, training time, model path, git commit hash and working-tree dirty flag.
- Dependencies are pinned in `requirements.txt`.
- The entire `experiments/` tree is regenerable with a single command.

6.3 Review outcomes

A strict pre-release review produced the following changes.

Issue	Severity	Resolution
Imputation fit on full population	Critical	fit/transform contract; regression tests
Train metrics never computed	High	all four splits evaluated; overfit warning
Results table re-read from disk	Medium	metrics carried in memory
Feature matrix re-converted per split	Medium	converted once
`assert` used for correctness	Medium	replaced with a typed exception
Model order depended on dict order	Medium	explicit ordering, sorted by run id
`sys.path` manipulation in scripts	Medium	proper package via `pyproject.toml`
No dependency pinning	Medium	`requirements.txt` with exact versions
Plot function coupled to table schema	Low	takes label/value sequences
Duplicated checks and cache loading	Low	extracted to shared helpers
Legacy scripts duplicating pipeline	Low	removed (history in git)
Unused imports and variables	Low	removed

Issue	Severity	Resolution

Table 5: Code review findings and resolutions.

6.4 Scale

Approximately 2,600 lines of Python across 34 modules, with 24 unit tests covering the invariants that matter: leakage prevention, patient-level split isolation, ensemble arithmetic and configuration validation. Twelve trained models and 59 generated figures.

7 Limitations

- **Biosignal modality unavailable.** The candidate pool is 12 models rather than the planned 16.
- **Overfitting.** Four models reach train AUC 1.000. Hyperparameter grids were kept small for a two-core environment and should be widened.
- **Single seed.** All results use seed 42. The ≈ 0.008 AUC ensemble margin has not been shown to exceed split-to-split noise.
- **SHAP for XGBoost.** `shap.TreeExplainer` fails on this shap/XGBoost version pair when parsing XGBoost's serialised `base_score`. The failure is caught and logged; feature importance is unaffected. This is a version-compatibility issue, not a defect in the pipeline.
- **No biospecimen modality.** The columns exist in the curated cut but carry 60–80% baseline missingness, requiring a deliberate missing-data decision before inclusion.

8 Recommendations for Month 2

8.1 Handover

Month 1 provides per-model predicted probabilities for all four splits on disk, an untouched calibration split of 211 patients, a single call that reconstructs every trained run, and an aggregation hook that accepts an arbitrary weight vector. Month 2 requires no retraining to begin.

8.2 A theoretical caveat that should be settled early

Magesh et al. detect whether *one input* is out-of-distribution by combining K statistics from *one model*. This project inverts that: it detects whether *one model* is an outlier by comparing M models across many inputs. The machinery — conformal p-values, step-up multiple testing, false-alarm control — transfers structurally, but the guarantee does not transfer automatically.

Their Theorem 2 bounds the false-alarm rate for a single test input against a calibration set of *exchangeable* inputs. In the per-model setting the objects under test are models, and twelve models trained on overlapping data are neither independent nor exchangeable in the required sense.

Two defensible outcomes exist, and which one holds should be established by week 6 rather than deferred to the writing phase:

1. **The guarantee transfers with modification**, for example by defining exchangeability over *patients* rather than models: for each calibration patient, compute each model's deviation from the leave-one-model-out consensus and treat those per-patient deviations as the exchangeable units. This is the more promising route.

2. **It does not transfer**, in which case the contribution is an empirically effective, principled-by-construction selection procedure benchmarked against the Month 1 baselines. This remains publishable; claiming a guarantee that does not hold would not be.

8.3 Suggested schedule

Weeks 5–6

Define and compare at least three model-level outlier scores: deviation from the leave-one-model-out consensus, mean pairwise disagreement, and calibration error. Sanity-check against a known weak model — clinical + NeuralNet, test AUC 0.707 — which any working score should flag.

Weeks 6–7

Compute conformal p-values on the calibration split, bootstrapping over patients. Verify empirically that p-values are approximately uniform under the null; this is where a broken exchangeability assumption will reveal itself.

Weeks 7–8

Apply the step-up procedure. Because model scores are strongly dependent, use Benjamini–Yekutieli rather than plain Benjamini–Hochberg. Aggregate survivors by trimmed mean or confidence-weighted average, and ablate removal versus down-weighting, the level α, and Bonferroni as a comparison.

8.4 Principal risks

- **The margin is thin.** A 0.008 AUC difference on 142 test patients is within plausible split noise. Run at least five seeds and report mean and standard deviation; lead with variance across conditions rather than mean performance, which is the claim the method is designed to support.
- **Pruning may not help when all models are competent.** Construct conditions where genuine outlier models exist — the five distribution-shift datasets already assembled — and verify that a deliberately degraded model (trained on shuffled labels) is removed. A method that cannot remove a label-shuffled model is not working, whatever its AUC.
- **Overfit models will distort scores.** Address regularisation before building selection logic on their validation behaviour.
- **Test-set discipline.** Every design decision must be made on the validation and calibration splits. The current code enforces this by convention rather than by construction.

9 Conclusion

Month 1 delivered a reproducible, configuration-driven baseline pipeline with a 12-model candidate pool and five ensemble baselines, together with the documentation, diagrams and tests needed for another researcher to reproduce and extend it. The most consequential outcomes were not the headline metrics but the two leakage problems identified and corrected: without those fixes, the reported test scores would have been optimistic in a way that would likely have survived into a submitted paper.

The baseline that Month 2 must beat is 0.9894 test ROC-AUC from simple averaging. That bar is high and the available margin is narrow, which is itself a useful finding to have established before the research contribution is built rather than after.